

Contrato API Semtex

Guía para el frontend: cómo conectarse al backend de Semtex.

- **Base URL local:** `http://localhost:8080`
- **Swagger UI:** `http://localhost:8080/swagger-ui.html` • **OpenAPI JSON:** `http://localhost:8080/v3/api-docs`

Autenticación y multi-tenant

Todas las rutas `/api/**` son privadas y requieren un JWT (Bearer) emitido por Supabase:

```
Authorization: Bearer <jwt>
Content-Type: application/json
```

El JWT debe incluir estos claims:

```
{
  "sub": "uuid-del-usuario",
  "email": "user@empresa.com",
  "org_id": "uuid-de-la-organizacion",
  "role": "ADMIN"
}
```

Importante (cambio de contrato): la organización y el usuario se derivan **siempre del token** (`org_id` y `sub`). El cliente **ya NO envía `organizationId` ni `userId`** en el body ni en los query params. El backend aplica el aislamiento por tenant de forma automática (filtro Hibernate por `organization_id`), así que un cliente nunca puede leer ni escribir datos de otra organización.

Roles

Rol	Puede
ADMIN	Todo: gestiona organización, usuarios, documentos, chat y lee auditoría.
OPERATOR	Sube documentos y usa el chat; lecturas.
AUDITOR	Lecturas + auditoría (no sube ni gestiona).

Permisos por endpoint indicados en cada sección. Las lecturas simples (detalle/listado) solo requieren estar autenticado.

Formato de error

Todos los errores devuelven el mismo cuerpo (campos nulos se omiten):

```
{
  "timestamp": "2026-06-18T16:00:00.123-05:00",
  "status": 400,
  "error": "Bad Request",
  "message": "La solicitud contiene campos inválidos.",
  "path": "/api/users",
  "fieldErrors": {
    "email": "El email no es válido"
  }
}
```

`fieldErrors` solo aparece en errores de validación (400).

Registro (alta de empresa) — `POST /api/register` · público (sin token)

Punto de entrada del sistema: crea una **organización** y su **primer usuario ADMIN** en una sola llamada. Es el único endpoint `/api/**` que no requiere JWT (resuelve el arranque, ya que crear usuarios/organizaciones por la API normal exige ser ADMIN).

```
{
  "organizationName": "Ferretería López",
  "slug": "ferreteria-lopez",
  "adminEmail": "dueno@ferrelopez.com",
  "adminUserId": "uuid-opcional-del-usuario-en-supabase"
}
```

`adminUserId` es **opcional**: en producción se pasa el sub del usuario ya registrado en Supabase Auth (para que el id coincida); si se omite, el backend genera uno. Respuesta 201:

```
{
  "organization": { "id": "uuid", "name": "Ferretería López", "slug": "ferre"
  "admin": { "id": "uuid", "email": "dueno@ferrelopez.com", "role": "ADMIN"
}
```

A partir de aquí, ese ADMIN ya puede autenticarse (con su JWT) y crear el resto de usuarios.

Organizaciones

Crear organización — POST /api/organizations · ADMIN

```
{ "name": "Ferretería López", "slug": "ferreteria-lopez" }
```

slug debe cumplir `^[a-z0-9-]{2,100}$`. Respuesta 201:

```
{
  "id": "uuid",
  "name": "Ferretería López",
  "slug": "ferreteria-lopez",
  "active": true,
  "createdAt": "2026-06-18T10:00:00",
  "updatedAt": "2026-06-18T10:00:00"
}
```

Obtener la organización del token — GET /api/organizations

Devuelve la organización del token (lista de un elemento por el filtro de tenant).

Detalle, renombrar y desactivar

- GET /api/organizations/{id}
- PATCH /api/organizations/{id} — **ADMIN** — body { "name": "Nuevo nombre" }
- DELETE /api/organizations/{id} — **ADMIN** — desactiva (204)

Usuarios

Crear usuario — POST /api/users · ADMIN

La organización se toma del token; **no** se envía organizationId.

```
{ "email": "operador@empresa.com", "role": "OPERATOR" }
```

Respuesta 201:

```
{
  "id": "uuid",
  "email": "operador@empresa.com",
  "role": "OPERATOR",
  "active": true,
  "lastLoginAt": null,
  "createdAt": "2026-06-18T10:00:00"
}
```

Listar usuarios — GET /api/users

Lista los usuarios de la organización del token (sin query params).

Gestión

- GET /api/users/{id}
 - PATCH /api/users/{id}/role — **ADMIN** — body { "role": "AUDITOR" }
 - DELETE /api/users/{id} — **ADMIN** — desactiva el usuario (204)
-

Documentos

Subir Excel/CSV — POST /api/documents · ADMIN / OPERATOR

multipart/form-data con un único campo file (.csv, .xlsx o .xls). La organización y el autor se derivan del token; **no** se envían organizationId ni uploadedByUserId.

```
curl -X POST http://localhost:8080/api/documents \  
  -H "Authorization: Bearer $TOKEN" \  
  -F "file=@balance.csv"
```

Respuesta 201:

```
{  
  "id": "uuid",  
  "name": "balance.csv",  
  "storagePath": "<org-id>/<uuid>-balance.csv",  
  "mimeType": "text/csv",  
  "fileSizeBytes": 12345,  
  "organizationId": "uuid",  
  "uploadedBy": "uuid",  
  "createdAt": "2026-06-18T10:00:00"  
}
```

El original se guarda en object storage (S3/MinIO) vía FileStoragePort; al subirlo se ingieren las filas como registros financieros.

Listado y detalle

- GET /api/documents — documentos de la organización del token (orden createdAt DESC)
 - GET /api/documents/{id}
 - DELETE /api/documents/{id} — **ADMIN** (204)
-

Registros financieros

Creados automáticamente al subir un documento. La organización sale del token.

Listar — GET `/api/financial-records`

Query params (todos opcionales):

- `documentId={uuid}` — filas de un documento concreto
- `fieldName=categoria&value=Ventas` — búsqueda exacta por campo JSONB (índice GIN)
- `limit=100` — por defecto 100; tope interno 1000

```
[
  {
    "id": "uuid",
    "documentId": "uuid",
    "sheetName": "default",
    "rowIndex": 1,
    "rowData": { "fecha": "2026-06-01", "categoria": "Ventas", "ingreso": 1000 },
    "createdAt": "2026-06-18T10:00:00"
  }
]
```

Chat IA

Enviar mensaje — POST `/api/chat/messages` · ADMIN / OPERATOR

Organización y usuario salen del token; **solo** se envía `content` y opcionalmente `documentId`.

```
{ "content": "¿Cuál fue el total de ventas?", "documentId": "uuid" }
```

`documentId` puede omitirse para una conversación sin contexto financiero. Respuesta:

```
{
  "agentResponse": "El total de ventas fue ...",
  "relevantRecords": [
    {
      "id": "uuid",
      "documentId": "uuid",
      "sheetName": "default",
      "rowIndex": 1,
      "rowData": { "categoria": "Ventas", "ingreso": 1000 },
      "createdAt": "2026-06-18T10:00:00"
    }
  ]
}
```

```
]
}
```

El agente (LangChain4j + Ollama, llama3.1:8b) puede invocar herramientas (consultarDatosFinancieros, compararPeriodos, enviarCorreo). El contexto financiero se limita con `semtex.ai.max-context-records`.

Historial — GET /api/chat/messages · ADMIN / OPERATOR

Query params: `documentId={uuid}` (opcional), `limit=100`. Devuelve los mensajes del usuario del token en orden cronológico:

```
[
  {
    "id": "uuid",
    "role": "USER",
    "content": "¿Cuál fue el total de ventas?",
    "userId": "uuid",
    "documentId": "uuid",
    "tokensUsed": null,
    "createdAt": "2026-06-18T10:00:00"
  }
]
```

`role ∈ USER | ASSISTANT`.

Auditoría

Listar — GET /api/audit/logs · ADMIN / AUDITOR

Query param: `limit=100`. La organización sale del token.

```
[
  {
    "id": "uuid",
    "action": "DOCUMENT_UPLOADED",
    "description": "Documento subido: balance.csv",
    "performedBy": "uuid",
    "createdAt": "2026-06-18T10:00:00"
  }
]
```

`action ∈ DOCUMENT_UPLOADED | FINANCIAL_QUERY | EMAIL_SENT | EMAIL_FAILED | USER_LOGIN | USER_CREATED | USER_DEACTIVATED | ROLE_CHANGED`.

Resumen de endpoints

Método	Ruta	Rol mínimo	Tenant
POST	/api/register	público	crea org + ADMIN
POST	/api/organizations	ADMIN	—
GET	/api/organizations/{id}	autenticado	token
PATCH/DELETE	/api/organizations/{id}	ADMIN	token
POST	/api/users	ADMIN	token
GET	/api/users/{id}	autenticado	token
PATCH /{id}/role · DELETE /{id}	/api/users	ADMIN	token
POST	/api/documents	ADMIN/ OPERATOR	token
GET	/api/documents/{id}	autenticado	token
DELETE	/api/documents/{id}	ADMIN	token
GET	/api/financial-records	autenticado	token
POST/GET	/api/chat/messages	ADMIN/ OPERATOR	token
GET	/api/audit/logs	ADMIN/ AUDITOR	token

Notas de implementación

- Aislamiento tenant **por defecto**: TenantContext request-scoped (poblado desde el JWT) + filtro Hibernate global por organization_id. El cliente nunca envía el tenant.
- Lecturas financieras combinan organizationId (del token) + documentId.
- Listados grandes usan limit con tope interno.
- JSONB con consulta @> para aprovechar el índice GIN en búsquedas exactas.
- Errores uniformes vía @RestControllerAdvice para el frontend.
- CORS configurable con semtex.cors.allowed-origins.